

Final Report – 00460882 Project

We chose the mdp and go benchmarks as described in the following chapters

Github repository: <https://github.com/AssafKlein-tech/HW-SW-Proj>

MDP Benchmark Overview

We selected the *mdp* benchmark, which models a turn-based Pokémon battle between two Pokémon, Charmander and Staryu, as a Markov Decision Process (MDP). The goal is to find the strategy that maximizes the probability of winning. The benchmark uses value iteration to compute the optimal winning probability and the policy that achieves it. It builds the MDP as a graph. Each node is a state, defined by both Pokémon's current HP and whose turn it is. Edges link states through the actions a player can take, and the resulting state depends on the damage each side takes.

The benchmark is CPU-bound. Its workload relies mainly on tuples, dictionaries, exact rational arithmetic with Python's Fraction type, and an explicit stack that replaces recursion. The explicit stack lets the benchmark explore deep state spaces without hitting Python's recursion limit.

Data structures. Each state is a collections.namedtuple that holds both Pokémon's stats, and each Pokémon's stats are also a tuple. The benchmark also uses collections.defaultdict in three places: to represent the probability distributions over successor states, to store two probability bounds for each state - a lower bound and an upper bound on its winning probability, and to store the successors tuple states.

The implemented Algorithm. The computation has two phases.

1. **State-space exploration.** A depth-first search, driven by an explicit stack, enumerates every state reachable from the initial state until it reaches a win or a loss. It also orders the states by their dependencies, which approximates a topological order from the win to the beginning through every state. The state graph is not acyclic, though. A healing move followed by incoming damage can return the battle to an earlier state, so this order removes the cycles, which takes effect only on the outer loop iteration.
2. **Value iteration.** Because of these cycles, the winning probabilities cannot be computed in a single backward pass. Instead, the benchmark runs a fixed-

point iteration. In each sweep, it visits the non-terminal states in the order found in phase 1, and recomputes each state's lower bound (dmin) and upper bound (dmax) from the current bounds of its successors. The bounds depend on the chosen action and the transition probabilities. Once a state's two bounds are equal, the state is frozen and skipped in later sweeps. The iteration stops when the lower and upper bounds of the initial state are within a given tolerance of each other. The converged value is the optimal probability of winning, and the actions that achieve it form the optimal strategy.

Pyperformance profiling

We created a script to run the performance. It emits the report for the flame graph but also added two additional profilers. A flat pref profile of the functions that the script runs. And cProfile runs against the main Battle().evaluate() call, creating a report of the function calls, including symbols for the benchmark, to analyze how much time was spent in each function and who called it.

Baseline measurement – over 20 runs, the run time was 10.5 sec +/- 0.7 sec per evaluate() call.

Initial Analysis and optimization suggestions

`_PyEval_EvalFrameDefault`

The interpreter's own bytecode dispatch loop - 13.29%. This is the unavoidable baseline cost of running Python at all.

Fraction operation:

fractions.py operations (`_add`, `forward`, `__new__`, `math.gcd`) account for ~24% of total runtime in the cProfile trace.

Fraction is used only to build intermediate probability distributions inside `getCritDist`, `_applyActionSide1`, `_getSuccessorsB` and `_getSuccessorsC`. Looking closely at those four functions, every one of them already converts its result to a plain float before returning it to its caller (`_getSuccessorsB` and `_getSuccessorsC` both end with `'float(p)'`), and the value-iteration loop's own accumulators (`dmin`, `dmax`) were already `'defaultdict(float)'` from the start. In other words, Fraction's exactness was being thrown away a few lines later in every single case. The

suggested optimization is to change all Fraction occurrences into float type. By doing that, we can lose some precision, but the results have a tolerance of $1e-6$.

getSuccessors and Hash function:

Profiling shows that about 40% of the benchmark's execution time is spent in `Battle.getSuccessors()`. During the initial topological-sort phase, this function generates every reachable state and caches each state's successors in a successors dictionary. During value iteration, it does no new computation. It only looks up a state in that dictionary and returns its list of successor states. The lookup key is the state itself, a nested tuple, so every lookup has to hash the whole nested structure. In a single `evaluate()` call, `getSuccessors()` was called 784,243 times. Much of that time is interpreter overhead: a full Python function call made only to perform one dictionary lookup.

To measure this overhead, we looked at how much time goes to hashing and dictionary operations. `PyObject_Hash` accounts for 9.96% of all samples and `tuplehash` for 8.37%, so these two functions alone make up 18.3% of the total. Behind them is a long tail of dictionary-related functions with the same cause, including `lookdict_unicode_nodummy` (3.25%), `lookdict` (2.50%), `long_hash` (2.12%) and `lookdict_unicode` (0.76%). With the help of an AI assistant (Claude), we added up every hash and dictionary-related symbol in the profiling report. Together they account for about 30% of all samples.

We then looked at the size of the state graph and found that it has only 4,823 nodes. A graph this small can easily be stored in an array-based representation. The proposed optimization gives each state a unique integer ID during the exploration phase. Each state's successors are then stored as a list of IDs, and its bounds (dmin, dmax) and its frozen flag are stored in lists indexed by the same ID. A successor lookup becomes a direct list index, which in CPython is a simple offset calculation instead of a hash of a nested tuple followed by a probe of the hash table. The successors of each state would be computed once, 4,823 times in total. This would remove about 779,420 redundant calls to `getSuccessors()` ($784,243 - 4,823$), along with the dictionary lookups on dmin, dmax and frozen that happen in every iteration.

Performance comparison

Optimization 1: replace fractions.Fraction with plain float

We replaced the Fraction operations with floats, resulting in 7.69 sec +/- 0.39 1.36x faster compared to the baseline, without any effect on the final result precision.

Optimization 2: replace state-tuple dict keys with integer-id-indexed lists

The implementation - `topoSort()` already visits every reachable state exactly once, in a fixed order, as a normal side effect of building the topological order we need anyway. So we used that existing order to assign each state an integer id, and changed all the dictionary data structures (dmin, dmax and frozen) to lists using the same id as index. Removing all of their hash and dict computing.

On the `topoSort`, we also added the successors into the successors list (indexed by id, containing id's of the successors). Now, instead of calling `getSuccessors()` we just access the list. The function call count dropped to 4821, exactly as we described.

The results after applying the two optimizations ended with 3.36 sec +/- 0.14 3.12x faster than the baseline.

The flat perf profile of the final version confirms the hash/dict machinery that used to dominate is largely gone: `PyObject_Hash` and `tuplehash` combined dropped from 18.3% of samples to under 3.5%, and `lookdict_unicode_nodummy` from 3.25% to under half that. `_PyEval_EvalFrameDefault` (the interpreter's own bytecode-dispatch overhead) is now the single largest cost, at 24% - which is essentially the practical floor for a pure-Python implementation of this algorithm; there is no more low-

hanging dict/tuple overhead left to remove without changing the algorithm or data representation more radically.

Conclusion

We start with a careful tool-cross-validated profile (perf flat profiling plus cProfile, since perf's native call-graph unwinding is unreliable under python3-dbg's LTO build). We identified two independent, low-risk optimization targets in the mdp benchmark we clearly saw in the flat profiling: Fraction arithmetic, and re-hashing the same nested-tuple state keys far more often than it needed to. Applying them incrementally - Fraction to float first, then nested tuple dicts to indexed lists + removing redundant function calls- let us measure each optimization's individual contribution rather than only the combined effect. The two together give a 3.12x (roughly 68%) speedup over baseline, with correctness preserved throughout (the benchmark's internal 1e-6 tolerance check passes at every stage).

GO Benchmark Overview

The second benchmark we selected is GO, a pure-Python Monte-Carlo Go engine. `versus_cpu()` seeds the RNG with 1, builds an empty 9x9 board and asks for ONE move. Choosing it costs 200 full simulated games, so the benchmark measures interpreter throughput on tight object-oriented pointer-chasing code - attribute access, method calls, small-int arithmetic, list indexing. It is deterministic: the answer is always the same.

Libraries - standard library only, no third-party dependencies:

- `random` playout move choice + Zobrist key generation
- `math` `sqrt/log` for the UCB1 exploration term
- `pyperf` timing harness only; not part of the workload

Data structures

- Square x81 - one board point. `color`, `ledges` (= liberty count), `reference` (union-find parent), `neighbours` (precomputed once)
- union-find - stone groups. `find()` walks reference to the group root; `move()` unions with same-colour neighbours, merging liberties.
- EmptySet - `empties[]` + `empty_pos[]`; O(1) add/remove by swap-with-last, and uniform sampling with in-place rejection.
- ZobristHash - incremental XOR board hash + a set of every hash ever seen-> O(1) superko detection.
- UCTNode - search tree; `pos_child` is a flat 81-slot list, plus wins/losses and a cached bestchild.

Algorithm

The benchmark chooses a single move on a 9×9 board by Monte-Carlo tree search: it runs 200 simulated games, each one replaying the current position from scratch and then descending the UCT tree - taking unexplored children first, otherwise the child cached as best under UCB1, $\text{winrate} + \sqrt{\log(\text{parent_visits}) / (5 \cdot \text{node_visits})}$ - until it expands one new node, from which it plays a uniformly random playout of up to 243 moves, scores the finished position by area with a komi of 7.5 to White, and propagates the win or loss back up the path it took. The machinery underneath each of those moves is where the time actually goes: stone groups are held as a union-find forest carrying a per-group liberty count, so playing a stone unions it with its same-colour neighbours, merges their liberties and decrements the enemy's, removing a group when its count reaches zero; empty points live in an index-plus-dense-array set that gives O(1) removal and uniform

sampling with in-place rejection; and every candidate point is first vetted by `useful()`, which speculatively plays the stone, counts friendly and enemy groups through `find()`, tests an incrementally XOR-updated Zobrist hash against the set of every position seen so far to reject superko repeats, and then rolls the hash back — relying on a global timestamp epoch to invalidate its scratch marks rather than clearing 81 flags per call.

Initial Analysis and optimization suggestions

Initial Analysis and optimization suggestions

We generated both a flame graph and cProfile run. `Square.find()` with 22.3% of running time and 463,922 calls is the dominant function, followed by `Board.useful()` with 19.7% of running time and 59,658 calls.

First thing that we observe is that `find()` is usually called in a deep recursively way. This can be observed visually in the flame graph, and further validated using the cProfile output, which shows only 183,082 outermost calls out of 463,922 total calls (2.53 calls per outermost call).

Second thing to observe is that when comparing two columns of the profile output: `tottime` (time spent exclusively executing lines inside the function) against `cumtime` (total time from when the function starts until it exits) for `find()` functions it is identical. When inspecting the actual code, we can observe that none of them is doing heavy arithmetic, allocation or I/O operations.

SW Optimization 1

The first optimization we utilize is converting `find()` function from being recursive to use a `while()` loop. The transition to `while()` loop is very simple since this function is simple to begin with, it retains the same computational work but removes the additional overhead of calling a function, that why we expect it to pay off.

SW Optimization 2

The second optimization we suggest is using `__slots__` in the `Square` class. By default every Python instance keeps its attributes in a per-instance dictionary `Square.__dict__`. `__slots__` is a class-level declaration that replaces that dict with fixed array of pointer-sized slots inside the object struct itself. Now each time that the code access an attribute of `Square` instance, instead of heavy dict access it only need to access it by the pointer specified. This optimization doesn't break anything because `Square` has no subclasses, and its dict is never being access directly. We expect this optimization to pay off because `Square` has 11 attributes that being access frequently.

Performance comparison

We first observe that the find() optimization works as predicted and number of calls went down to 183,082. We then profile each optimization separately and together and measure runtime over 15 repetitions, those are the results:

	Mean runtime	Speedup
Baseline	394.1 ms	1.000 x
Optimization 1	376.2 ms	1.048 x
Optimization 2	360.8 ms	1.092 x
Combined	339.8 ms	1.160 x

GO HW optimization

We mentioned before that the most dominated functions are `find()` with 22.3% of runtime and `useful()` with 19.7% of runtime. `Find()` might look as a better candidate for offloading to accelerator, but it isn't for 2 reasons:

1. It being called much more - 183,082 outermost calls against only 59,658 calls. When offloading to accelerator, we save time by faster implementation on hardware, but we pay with bus roundtrip for every call.
2. When counting total time from when the function starts until it exits, `useful()` has bigger share because it calls other functions. If the entire logic of `useful()` is implemented on hardware we will save more time.

In addition, `useful()` functionality will suit a hardware acceleration. In the software implementation it includes mainly pointer hops, int comparison and 63 bits XOR. Its state is small (the entire board is 18Kbts) and can fit in on cheap memory, and it is not being updated frequently.

The numbers behind that choice, from the flame graph and cProfile:

	find()	useful()
Calls per iteration	183,082	59,658
Runtime it owns (inclusive)	87 ms (22%)	185 ms (47%)
Budget per bus round trip	0.48 μ s	3.1 μ s

A round trip to an accelerator costs around 1 μ s. Offloading `find()` would make the benchmark slower. Offloading `useful()` leaves room to win, and it takes 69% of the `find()` calls with it, since they happen inside `useful()`.

HW Architecture

Inputs:

Signal	Width	What it does
clk	1	Single clock, 100 MHz target.
rst_n	1	Synchronous, active low. Also sets <code>ref[i] = i</code> , so every point starts as its own group.
cmd_valid	1 (each)	Handshake. <code>cmd_ready</code> is asserted only when the core is idle — one operation in flight, no queue.
cmd_op	4	Which of five operations to run.

cmd_pos	7	The board point, 0...80 ($y*9 + x$). Seven bits because $\lceil \log_2 81 \rceil = 7$. For USEFUL it's the point being tested; for MOVE the point being played.
cmd_color	2	Whose move — EMPTY=0, WHITE=1, BLACK=2.
Cmd_key	63	Zobrist key payload for LOAD_KEY. 63, not 64, because the software draws <code>random.randrange(2**63 - 1)</code> .
cmd_key_idx	9	Where to file that key: $\text{square} * 3 + \text{colour}$, so 243 live entries.

Outputs:

Signal	Width	What it does
busy	1	State is not idle
Done	1	One-cycle completion pulse
Res_fast	1	The USEFUL took the fast path — an untouched point with an empty neighbour. When set, the answer is simply yes, and software must not run the superko test.
Res_cond	1	The predicate itself: $\text{empties} \vee \text{weak_opps} \vee (\text{strong_neighs} \wedge (\text{strong_opps} \vee \text{weak_neighs}))$. This is the whole of <code>useful()</code> except <code>superko</code> .
Res_hash	63	The candidate whole-board hash if the move were played, captures included. This is what software probes against its superko set. Hardwired to 0 when <code>res_fast</code> is 1.
Cur_hash	63	The committed board hash after the last MOVE or RESET.

Opcodes:

OP	Inputs used	What it does	outputs
NOP (0)			done
Reset (1)		empty the board, $\text{ref}[i]=i$, then rebuild the hash by XOR-ing the 81 EMPTY keys	cur_hash
Move (2)	cmd_pos, cmd_color	commit a stone: union with friendly groups, decrement enemy liberties, cascade-remove any group that hits zero, update the hash	done

Useful (3)	cmd_pos, cmd_color	the speculative predicate; writes no committed state	res_fast, res_cond, res_hash
Load_key (4)	Cmd_key, cmd_key_idx	store one Zobrist key	done

Register map:

Offset	Name	Dir	Fields
0x00	CMD	W	[31:28] op, [8:7] color, [6:0] pos. Writing it starts the operation.
0x04	STAT	R	busy, done, useful_cond, fast_path
0x08	RES_LO	R	hash [31:0]
0x0C	RES_HI	R	hash [62:32], bit 31 = done (one register to poll)
0x10 / 0x14	KEY_LO / KEY_HI	W	key [31:0] / key [62:32]
0x18	KEY_IDX	W	[8:0] = square*3 + colour. Writing it stores the key.

Software side:

- USEFUL is one write, a poll and two reads. MOVE is one posted write with no reply, so the CPU never waits for it.
- The Python board stays the master copy; the accelerator only mirrors it. The superko hash set stays in software because it is unbounded.
- The registers are memory-mapped. Python reaches them through a small shim, either ctypes over an mmap of the device or a C extension. The C extension is the faster of the two, about 1 μ s per round trip against 1.5 μ s.
- The whole change to the benchmark is about 15 lines. The new Board.useful() is:

```
fast, cond, h = ACCEL.useful(pos, self.color)

if fast: return True

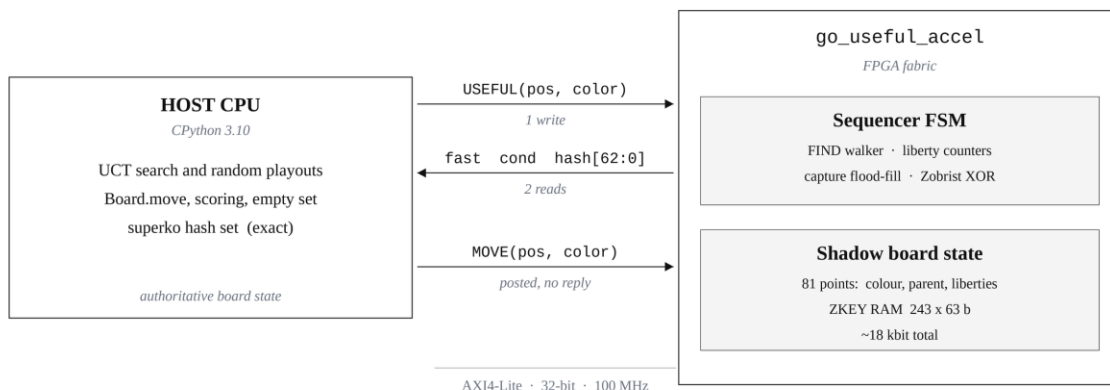
return cond and h not in self.zobrist.hash_set
```

Inside the block. The design is written in SystemVerilog: hw/rtl/go_useful_core.sv is the engine, hw/rtl/go_useful_accel.sv the AXI4-Lite wrapper, and hw/tb/ holds a self-checking testbench.

- A mirror of the board: colour, parent pointer, liberty count and used flag for the 81 points, two scratch bitmaps, the 243 Zobrist keys and the board hash. About 18 kbit, all on chip. Only the key table is a block RAM.
- One 24-state FSM with two shared sub-routines: FIND (one parent hop per cycle) and an XOR step that folds two keys into the hash.
- MOVE and USEFUL are the same neighbour walk. MOVE writes the result back, USEFUL only computes it.
- No pipelining. A USEFUL takes 2 to 30 cycles and a bus round trip about 100, so the block is idle most of the time anyway.

How a USEFUL runs:

1. Clear the two scratch bitmaps (1 cycle).
2. Fast path: if the point was never played and has an empty neighbour, answer yes and stop. Half of all calls end here, in 2 cycles.
3. Start the candidate hash: current hash XOR the two keys of this point.
4. For each of the up to 4 neighbours: if empty, count a liberty; otherwise FIND its group root, take one liberty off that group, and count it as friendly or enemy.
5. If an enemy group just lost its last liberty, flood-fill it and XOR its stones out of the candidate hash. This is the capture, done speculatively.
6. Return fast, cond and the candidate hash. Nothing in the board state was written.



Block diagram.

Expected gains and tradeoffs

Cycle counts, measured in simulation over one full benchmark iteration:

Operation	Count	Min	Mean	Max
USEFUL, fast path	28,199	2	2.0	2
USEFUL, full path	31,459	11	29.4	553
USEFUL, all	59,658	2	16.5	553
MOVE	21,401	11	28.4	655

A mean USEFUL is 0.165 μ s at 100 MHz, against about 3.1 μ s in Python. The block is not the bottleneck, the bus is. The runtime we can expect depends on the round trip:

Round trip	Integration	Runtime	Speedup
0.5 μ s	tightly coupled fabric	239 ms	1.63x
1.0 μ s	AXI4-Lite, C extension shim	268 ms	1.45x
1.5 μ s	AXI4-Lite, ctypes shim	298 ms	1.30x
3.1 μ s	break-even	392 ms	1.00x

- Speedup: about 1.45x expected. The ceiling is 1.86x, since useful() is 47% of the runtime and the rest stays in Python.
- Area: about 1.6k flip-flops, one block RAM and 1.5k LUTs. Under 2% of a mid-range FPGA.
- Frequency: the critical path is one FIND hop, so 200 MHz is possible. It would not help; 0.165 μ s of work already sits inside a 1 μ s round trip.
- Power: tens of mW, and the block is idle 80% of the time. Total system power still goes up, because the host busy-polls while it waits.
- Caveats: we have no FPGA, so the runtimes above are projected from measured cycle counts and an assumed round trip. The worst USEFUL is 553 cycles (5.5 μ s), above the 3.1 μ s budget, but that is 31 calls out of 59,658.

Verification

- A Python model of the accelerator, run inside the benchmark, gives a bit-identical result: same move, same 44,382 random draws, same 200 games.
- All 81,747 operations of one iteration were recorded and replayed through the RTL. Pass.
- Three bugs were injected into the RTL to check the testbench can fail. All three were caught.

